

Indexing Techniques for DW

B-Tree Indexes:

Query 1: Aggregates trade data for each user and asset in 2025, calculates total and average trade value, and returns the top 100 trades.

Without using Indexes:

31	Planning Time: 2.532 ms
32	Execution Time: 1.807 ms

With Indexes:

35	Heap Fetches: 0
36	Planning Time: 18.654 ms
37	Execution Time: 4.717 ms

Query 2: Counts and sums total trades for specific users, showing their total trade volume.

Without using Indexes:

19	Planning Time: 0.411 ms
20	Execution Time: 0.523 ms
Total rows: 20 Query complete 00:00:00.100	

✓ Successfully run. Total que

Using Indexes:

15	Planning Time: 0.424 ms
16	Execution Time: 0.152 ms

Bitmap Index:

Query 3: Filters trades by low-cardinality column risk_profile, demonstrating potential bitmap index benefit.

Without using Indexes:

9	Planning Time: 0.359 ms
10	Execution Time: 0.230 ms

Using Indexes:

9	Planning Time: 2.257 ms
10	Execution Time: 0.123 ms

Materialized Views or Cached Aggregations

Query 4: Aggregates monthly trade stats per user and asset for analysis.

Without making Materialized View:

29	Planning Time: 2.437 ms
30	Execution Time: 13.184 ms

Using Materialized View:

	QUERY PLAN text
1	Seq Scan on mv_trade_summary (cost=0.00..25.75 rows=975 width=68) (actual time=0.024..0.159 rows=975 loops=...
2	Planning Time: 0.074 ms
3	Execution Time: 0.225 ms

Table comparing index type, purpose, and runtime improvement.

Index Type	Attribute(s)	Purpose	Execution time (before)	Execution time (after)	Runtime Improvement (%)	Observation
B-Tree	date_id, symbol, user_key	Speeds up joins & filtering on key columns	4.717	1.807	62%	Effective for multi-column joins; planner used Index Scan instead of Seq Scan
B-Tree	user_key	Improves user-level lookups & joins	0.523	0.152	71%	Great for filtering specific users; high selectivity
Bitmap	risk_profile	Low-cardinality filter	0.230	0.123	47%	Planner used bitmap scan internally; efficient for few distinct values
Materialized View	monthly trade stats	Cache aggregated data	13.184	0.225	98%	Huge speedup; avoids repeated complex aggregation. Just need of read cost

Discussion on why certain joins or indexes are best suited for DSS workloads (versus OLTP).

Indexes:

- **B-Tree:** Best for high-cardinality columns, range queries, and joins. Improves time-based or user-level lookups, e.g., date_id, user_key.
- **Bitmap:** Efficient for low-cardinality attributes, e.g., risk_profile. Reduces I/O for filtering large fact tables in DSS.
- **Materialized Views:** Pre-aggregated results for frequent queries, dramatically speeding up DSS workloads.

Join Methods:

- **Nested Loop Join (NLJ):** Simple, suitable for small tables or indexed joins. It is common in OLTP.
- **Hash Join:** Efficient for large, unsorted tables. Builds hash tables for quick matching and ideal for DSS bulk joins.
- **Sort-Merge Join:** Requires sorted inputs, good for large datasets and range queries, often used in DSS where indexed or pre-sorted data is available.

Summary:

DSS workloads benefit from B-Tree and bitmap indexes and hash or sort-merge joins due to large tables and aggregations. OLTP workloads, with smaller, transactional tables, often rely on NLJ and simpler indexes. Materialized views further enhance DSS query performance by caching aggregated data.